API Data Flow Reference

System overview

PetPals uses a two-token authentication system. Authentication credentials are centralised in a single USERS table shared across all six role-specific tables (Admin, Staff, Veterinarian, Adopter, Volunteer, Donor).

Token	Storage	Expiry	Purpose
Access token	Zustand memory (frontend)	15 minutes	Sent in Authorization header on every protected API request. For Shelter Staff, payload also includes shelterID (see 1.2 and 1.4).
Refresh token	httpOnly cookie (browser)	7 days	Used only to issue new access tokens - never sent in API headers

Architecture pattern followed by every backend endpoint:
Route → Middleware (protected routes only) → Controller → Service → Database

Frontend auth pattern on every page load:
App.tsx mounts → refreshToken() called → httpOnly cookie sent → new access token returned → Zustand populated

1. Backend Auth API

Base path: /api/v1/auth
Four endpoints covering registration, login, logout, and token refresh. Register and login are public. Logout requires a valid Bearer access token. Refresh-token requires only the httpOnly cookie.

1.1 POST /api/v1/auth/register

Creates a new user account. Writes two rows atomically - one to USERS (credentials) and one to the role-specific table (name + userID). The Prisma $transaction ensures both succeed or both roll back.

Request summary

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	name, email, password, role
Valid roles	admin, adopter, staff, vet, volunteer, donor
Success	201 - { userID, userEmail, role }

Data flow

File / Layer	What happens here
auth.routes.js	Receives POST request, no middleware - public route. Maps POST /register → register controller

auth.controller.js	Validates all 4 fields are present and role is one of the 6 valid values. Destructures { name, email, password, role } from req.body. Returns VALIDATION_ERROR (422) if any field is missing. Returns VALIDATION_ERROR (422) if role is not valid. Calls authService.register() in try/catch. On success: successResponse 201 with created user. On failure: next(err) → errorHandler
auth.service.js	Duplicate email check, bcrypt hash, atomic double-insert via Prisma transaction. findUnique on USERS by email - throws CONFLICT (409) if exists. bcrypt.hash(password, 10). Opens $transaction: creates USERS row, then role-specific row using returned userID. If either insert fails, entire transaction rolls back - no orphaned records. Strips userPassword from returned object
USERS table	Stores auth credentials. Columns written: userEmail, userPassword (hashed), role
Role table	Stores name and userID for the specific role. Columns written: userID (FK → USERS), [role]Name (e.g. adopterName)

Error cases

Scenario	Error Code	Response Message
Missing any of the 4 fields	VALIDATION_ERROR	name, email, password, and role are required
Role not in the allowed list	VALIDATION_ERROR	role must be one of: admin, adopter, staff, vet, volunteer, donor
Email already registered	CONFLICT	A user with this email is already registered
DB insert fails	INTERNAL_SERVER_ERROR	Internal server error

1.2 POST /api/v1/auth/login

Authenticates an existing user. Verifies credentials against USERS, issues a 15m access token in the response body and a 7d refresh token in an httpOnly cookie. Also fetches the user's name from the role-specific table for UI display. For Shelter Staff, the access token payload also embeds the staff member's current shelterID.

Request summary

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	email, password
Success	200 - { token (15m), user: { userID, userEmail, role, name } } + refreshToken httpOnly cookie (7d)

Data flow

File / Layer	What happens here
auth.routes.js	Receives POST request, no middleware - public route. Maps POST /login → login controller

auth.controller.js	Validates fields, calls service, signs both tokens, sets cookie. Returns VALIDATION_ERROR (422) if email or password missing. Calls authService.login() in try/catch. Signs access token payload: { userID, role } for all roles, plus shelterID when role === 'Staff' (value returned by authService.login()), 15m expiry, JWT_SECRET. Signs refresh token: { userID }, 7d expiry, JWT_SECRET. Hashes refresh token with bcrypt and calls authService.storeRefreshToken(userID, hashedRT). Sets refreshToken as httpOnly cookie (sameSite: strict, secure in production). Returns 200 with { token: accessToken, user }
auth.service.js (login)	Verifies credentials, fetches name from role table, returns safe user object. findUnique on USERS by email (selects userID, userEmail, role, userPassword only). Throws NOT_FOUND (404) if no record. bcrypt.compare(password, user.userPassword) - throws UNAUTHORIZED (401) if no match. Queries role-specific table for the user's name using ROLE_CONFIG. When role === 'Staff', the same query also selects shelterID and includes it in the returned object so the controller can embed it in the access token payload. Strips userPassword before returning { ...safeUser, name, shelterID? }
auth.service.js (storeRefreshToken)	Stores hashed refresh token in USERS table. prisma.users.update({ where: { userID }, data: { refreshToken: hashedRT } })
USERS table	Queried for credentials, updated with refresh token hash. Columns read: userEmail, userPassword. Columns written: refreshToken (hashed)
Role table	Queried for the user's display name. e.g. Adopter.adopterName, Staff.staffName etc. For Staff specifically, shelterID is also selected in this query.

Design note - shelterID scope: shelterID is included in the access token payload only for the Staff role. Other roles (Adopter, Vet, Volunteer, Donor, Admin) do not carry a shelterID claim, since branch-scoping only applies to Shelter Staff (see 02_Requirements S-12). This keeps the token payload minimal for roles that don't need it.

Error cases

Scenario	Error Code	Response Message
Missing email or password	VALIDATION_ERROR	email and password are required
Email not found in USERS	NOT_FOUND	No account found with this email
Password does not match hash	UNAUTHORIZED	Incorrect password

1.3 POST /api/v1/auth/logout

Ends the session by nullifying the stored refresh token in the DB and clearing the httpOnly cookie. The access token is not denylisted - with a 15-minute expiry the risk window is acceptable. The refresh token nullification is the source of truth for session invalidation.

Request summary

Field	Value
Method	POST
Auth required	Yes - Bearer access token in Authorization header
Body fields	None
Success	200 - { message: "Logged out successfully" }

Data flow

File / Layer	What happens here
auth.routes.js	Registers authenticate middleware before controller. Maps POST /logout to: authenticate → logout controller
authenticate.js	Validates the Bearer access token. Extracts token from Authorization header via optional chaining. Returns UNAUTHORIZED if missing. jwt.verify(token, JWT_SECRET) - throws if invalid or expired. Attaches { userID, role } to req.user and calls next()
auth.controller.js	Calls service with userID and clears the cookie. Uses req.user.userID (set by authenticate middleware). Calls authService.logout(userID) in try/catch. res.clearCookie('refreshToken') - removes httpOnly cookie from browser. Returns 200 with 'Logged out successfully'
auth.service.js	Nullifies stored refresh token in DB. prisma.users.update({ where: { userID }, data: { refreshToken: null } }). Any future refresh attempt will fail bcrypt comparison
USERS table	refreshToken column set to null. Columns written: refreshToken = null

Error cases

Scenario	Error Code	Response Message
No Authorization header	UNAUTHORIZED	Authentication failed - no token provided
Access token invalid or expired	UNAUTHORIZED	Authentication failed - invalid token

1.4 POST /api/v1/auth/refresh-token

Issues a new access token and rotates the refresh token. Does not require an access token - authentication is via the httpOnly cookie only. Called by the frontend on every app load to restore session from the cookie. For Shelter Staff, this is also the point where a stale shelterID claim self-corrects (see design note below).

Request summary

Field	Value
Method	POST
Auth required	No - cookie only (no Bearer token needed)
Cookie required	refreshToken httpOnly cookie
Body fields	None
Success	200 - { token (new access token, 15m), user: { userID, role, name } } + new refreshToken cookie (7d)

Data flow

File / Layer	What happens here
auth.routes.js	No middleware - cookie-only authentication handled in controller. Maps POST /refresh-token → refreshToken controller directly

auth.controller.js	Reads cookie, decodes userID from it, signs new tokens, calls service. Reads rawOldRT from req.cookies.refreshToken - returns UNAUTHORIZED if missing. jwt.decode(rawOldRT) to extract userID (no signature verification here). Returns UNAUTHORIZED if decoded.userID is absent. Signs new refresh token: { userID }, 7d expiry. Calls authService.refreshToken(userID, rawOldRT, newRefreshToken) in try/catch. On success: signs new access token payload { userID, role }, plus shelterID when role === 'Staff' (value returned fresh from authService.refreshToken(), not carried over from the old token), 15m expiry. Sets new refreshToken as httpOnly cookie. Returns 200 with new access token and { userID, role, name }
auth.service.js	Verifies old refresh token against DB hash, stores new hash, fetches name and current shelterID. findUnique on USERS by userID (selects userID, role, refreshToken). Throws NOT_FOUND if user missing. Throws UNAUTHORIZED if refreshToken in DB is null (already logged out). bcrypt.compare(rawOldRT, user.refreshToken) - throws UNAUTHORIZED if no match. bcrypt.hash(rawNewRT, 10) and updates USERS.refreshToken with new hash. Queries role-specific table for the user's name using ROLE_CONFIG. When role === 'Staff', this query re-fetches shelterID fresh from the STAFF table (not from the old token) - this is the mechanism that picks up a shelter reassignment. Returns { userID, role, name, shelterID? } (refreshToken stripped)
USERS table	refreshToken read for verification, updated with new hash. Columns read: refreshToken (hash for bcrypt comparison), role. Columns written: refreshToken (new hash)
STAFF table	Queried fresh on every refresh when role === 'Staff', to re-fetch the current shelterID. Columns read: shelterID

Design note - shelterID staleness and self-correction: the access token is short-lived (15 minutes), so if an admin reassigns a staff member to a different shelter branch, the staff member's current access token keeps the old shelterID until it expires - a small, bounded window. When the access token expires, the frontend's response interceptor triggers a silent call to /auth/refresh-token. Because that endpoint re-queries the STAFF table for shelterID rather than re-signing the old payload, the new access token reflects the updated shelter assignment automatically. No manual logout/login is required for the change to take effect - it self-corrects on the next refresh cycle.

Error cases

Scenario	Error Code	Response Message
No refreshToken cookie	UNAUTHORIZED	No refresh token provided
Cookie cannot be decoded	UNAUTHORIZED	Invalid refresh token
User not found in DB	NOT_FOUND	User not found
refreshToken null in DB (logged out)	UNAUTHORIZED	Session expired, please log in again
Cookie does not match DB hash	UNAUTHORIZED	Invalid refresh token

Design note - rotation

Each refresh token can only be used once. After a successful refresh the old hash is replaced in the DB. If an attacker steals a refresh token and tries to use it after the legitimate user has already rotated it, bcrypt.compare will fail and the request is rejected.

2. Backend Supporting Files

Shared across all auth endpoints

File	Purpose
config/prisma.js	Singleton Prisma client using PrismaPg adapter. Single gateway to the DB - imported by all service files.
utils/response.js	successResponse() and errorResponse() - standardise all API response shapes.
utils/errors.js	ERROR_CODES map - defines valid error codes and their HTTP status codes (400, 401, 403, 404, 409, 422, 500).
middleware/errorHandler.js	Global error handler registered last in app.js. Receives errors via next(err), maps err.code to HTTP status, formats using errorResponse().
middleware/authenticate.js	JWT verification middleware. Extracts Bearer token, verifies signature, attaches { userID, role } and, when present in the token payload, shelterID to req.user. Used on logout. Not used on refresh-token.
middleware/authorizeRoles.js	RBAC middleware. Takes ...allowedRoles, checks req.user.role against the list, returns 403 FORBIDDEN if not permitted. Applied to role-specific endpoints in
app.js	Express app setup - middleware, routes, error handler. No app.listen() call. Imported by tests directly.
index.js	Thin entry point - requires app.js and calls app.listen(). Split from app.js to allow Supertest to import the app without binding a port.

3. Backend Supporting Files — client/src | Sprint 1

The frontend auth system is built around four files that each have a single responsibility. They are intentionally decoupled - the API layer knows nothing about state, the state layer knows nothing about HTTP, and the UI layer composes both.

3.1 File responsibilities

File	Responsibility
api/authApi.ts	API functions for auth endpoints - register(), login(), refreshToken(), logout(). Each makes an axios call and returns typed data. Knows nothing about state.
api/axiosInstance.ts	Pre-configured axios client. Sets baseURL from VITE_API_URL env variable, withCredentials: true (sends httpOnly cookie automatically). Request interceptor attaches Bearer token from Zustand. Response interceptor handles 401 (clear state + redirect to /login) and 403 (redirect to /forbidden), skipping auth endpoints to avoid redirect loops.
store/useAuthStore.ts	Zustand global store. Holds { user, token, role } in memory - user now also carries shelterID for Staff sessions. login() sets all fields, logout() clears them. State is lost on page refresh - restored via refreshToken() on app load.
App.tsx	Root component. On mount, calls refreshToken() to restore session from the httpOnly cookie. On success, populates Zustand. On failure, does nothing - user will see public pages or be redirected to /login by ProtectedRoute. Uses authReady state to prevent flash of unauthenticated content.

3.2 Session restore on page load

Because Zustand state is memory-only, it is lost on every page refresh. The session is restored using the httpOnly cookie which persists across refreshes.

Flow

File / Layer	What happens here
App.tsx	useEffect fires on mount, calls refreshToken(). authReady state starts false - renders a rose background placeholder to prevent flash. refreshToken() called - axiosInstance sends the request with credentials: true. Browser automatically attaches the httpOnly refreshToken cookie
axiosInstance.ts	Sends POST /auth/refresh-token with withCredentials: true. isAuthCall guard on response interceptor prevents redirect loop if this call returns 401
Backend /auth/refresh-token	Verifies cookie, rotates tokens, re-fetches shelterID for Staff, returns new access token + name. New refreshToken set as httpOnly cookie in response. Returns { token, user: { userID, role, name, shelterID? } }
App.tsx .then()	On success, populates Zustand and sets authReady = true. storeLogin(user, token, user.role) - stores session in memory. authReady = true - full app renders, Navbar shows user name and dropdown
App.tsx .catch()	On failure (no cookie, expired), silently ignored. authReady = true - app renders normally. User sees public pages; ProtectedRoute redirects to /login if they try to access a dashboard

3.3 Login flow (Login.tsx)

File / Layer	What happens here
Login.tsx	Validates fields client-side, calls authApi.login(), updates Zustand, navigates. emailRegex validation + required field check before hitting API. noValidate on form - browser validation suppressed, custom error box used. On success: storeLogin(user, token, user.role) then navigate(`/${role.toLowerCase()}`, { replace: true }). replace: true prevents back button returning to login page. On failure: displays server error message inline. If user already authenticated (token in Zustand): <Navigate> redirect away from login immediately
authApi.ts (login)	POSTs { email, password } to /auth/login. Returns { token, user: { userID, userEmail, role, name, shelterID? } }. axiosInstance automatically receives and stores the httpOnly refreshToken cookie

3.4 Logout flow (Navbar.tsx)

File / Layer	What happens here
Navbar.tsx (handleLogout)	Calls API, clears Zustand, navigates to home. await logoutApi() - sends Bearer token, server nullifies DB refresh token and clears cookie. finally block: storeLogout() clears Zustand regardless of API success/failure. navigate('/') - user lands on home page
authApi.ts (logout)	POSTs to /auth/logout with Bearer token from Zustand. axiosInstance request interceptor attaches Authorization: Bearer <token> automatically

4. Role-Based Access Control — Frontend guards + backend middleware

PetPals enforces RBAC at two layers - the frontend prevents unauthenticated or wrong-role navigation, and the backend enforces the same rules on every API request. Both must be in place: frontend guards are UX, backend enforcement is security.

4.1 Frontend route guards

ProtectedRoute

Wraps any route that requires authentication. Reads token from Zustand - if absent, redirects to /login. If present, renders children.

Usage: <ProtectedRoute><AdopterDashboard /></ProtectedRoute>

RoleRoute

Wraps routes that require a specific role. Accepts an allowedRoles prop. Reads role from Zustand - if role is not in the allowed list, redirects to /forbidden. Used inside ProtectedRoute.

Usage: <ProtectedRoute><RoleRoute allowedRoles={['Adopter']}><AdopterDashboard /></RoleRoute></ProtectedRoute>

Route configuration in App.tsx

Route	ProtectedRoute	RoleRoute allowedRoles
/adopter	Yes	Adopter
/staff	Yes	Staff
/vet	Yes	Veterinarian
/volunteer	Yes	Volunteer
/donor	Yes	Donor
/admin	Yes	Admin
/login	No (redirects if already logged in)	-
/register	No (redirects if already logged in)	-
/forbidden	No	-

4.2 Backend RBAC middleware

authorizeRoles.js

Factory middleware that accepts a list of allowed roles and returns an Express middleware function. Called after authenticate so req.user.role is already set.

Usage: router.post('/pets', authenticate, authorizeRoles('Staff', 'Admin'), createPet)

Status in Sprint 1: middleware implemented and unit tested. Not yet applied to any routes - all Sprint 1 auth endpoints are either public or open to all authenticated users. Will be applied to resource endpoints starting Sprint 2.

ROLES constant

Key	Value (DB enum)
ROLES.ADMIN	Admin
ROLES.STAFF	Staff
ROLES.ADOPTER	Adopter
ROLES.VETERINARIAN	Veterinarian

ROLES.VOLUNTEER	Volunteer
ROLES.DONOR	Donor

4.3 Forbidden page

A simple public page at /forbidden rendered when RoleRoute or the axiosInstance 403 handler redirects an authenticated user who lacks the required role. Uses the same background image and card style as the login page.
Triggered by: RoleRoute (wrong role on frontend route) and axiosInstance response interceptor (403 from any API call).

5. Auth System Tests — Jest + Supertest | Sprint 1

Test file	Type	Cases covered
tests/unit/authenticate.test.js	Unit	No token → 401, Invalid JWT → 401, Expired JWT → 401, Valid JWT → sets req.user and calls next()
tests/unit/authorizeRoles.test.js	Unit	Role in allowedRoles → next(), Role NOT in allowedRoles → 403, Multiple allowed roles each pass
tests/integration/ auth.register.test.js	Integration	Valid payload → 201 + DB rows created, Missing fields → 422, Duplicate email → 409, Password hashed in DB
tests/integration/auth.login.test.js	Integration	Valid credentials → 200 + JWT, Wrong password → 401, Unregistered email → 404, JWT contains correct userID and role, Staff login JWT contains shelterID matching STAFF table
tests/integration/auth.logout.test.js	Integration	Valid token → 200 + DB refreshToken null, Stale refresh token after logout → 401, No token → 401
tests/integration/ auth.refreshToken.test.js	Integration	Valid cookie → 200 + new JWT, Staff refresh re-fetches current shelterID from STAFF table (not from old token), Staff reassigned to new shelter mid-session → next refresh reflects new shelterID, Non-staff role → no shelterID claim in new token

All integration tests seed data via the real register/login endpoints and clean up via Prisma in afterAll. Tests run sequentially (--runInBand) to avoid FK conflicts on the shared Supabase dev DB.